University Administrative Copilot

Comprehensive Technical and Integration Report

Ask the Copilot | Elective Advisor | Partner Integration

Group 10 | Roll No. J054, J055, J056, J057, J059

NMIMS administrative NLP application

Executive summary

The University Administrative Copilot is an NLP-backed FastAPI application for students and faculty who ask administrative questions in natural language. It classifies the request, extracts useful entities, retrieves official NMIMS evidence, produces a grounded answer, routes the request to a department, and optionally creates a structured service request. The same application contains the Group 10 elective-advisor workflow: multiple formal or free-text syllabi plus a student's programme and academic stage are converted into eligibility checks, ranked recommendations, citations, and a human-review handoff.

This report documents the implementation that is actually present in the repository. It does not claim a live university production deployment. The trained model binaries are gitignored and absent from the repository snapshot; a deterministic browser fallback is used when those artifacts are unavailable. Partner integration is contract-ready and mock-backed unless the two partner URLs are configured.

Table 1. Scope and evidence

Area	Implemented behavior	Evidence / boundary
Ask the copilot	Intent, sub-intent routing, NER, hybrid retrieval, extractive answer, service request, confidence decision	FastAPI /v1 endpoints and reusable typed component
Elective advisor	Parse syllabi, resolve programme/semester, apply published rules, rank eligible courses, cite checks	Week 7 saved evaluation: 98.46% held-out status accuracy; 12/12 recommendations
Partner integration	Consumes syllabus and student-record HTTP contracts; records calls; uses bundled mocks by default	Environment variables PARTNER_SYLLABUS_URL and PARTNER_STUDENT_URL
Fresh checkout	Browser remains usable through DemoCopilot when trained artifacts cannot load	Fallback is not equivalent to trained v3 inference

1. Problem statement and university context

The project brief describes a university in which administrative information is distributed across documents and systems. A student may ask, "I lost my ID card. What should I do?", while the answer requires intent classification, named-entity recognition, information extraction, semantic search and retrieval-augmented generation. The expected administrative result is not only prose: it identifies the service, responsible department, required documents and next action. The stretch goal is to generate a structured service request.

Group 10's elective problem is a second concrete administrative workflow. Given several syllabi on a topic, including a new course, and a student identity such as third year and Computer Science, the system identifies courses the student can enrol in. In a real university application, these two workflows would sit behind a student portal or helpdesk: the portal supplies the student's context, the copilot supplies grounded guidance, and a staff member remains the authority for enrolment and exceptional cases.

1.1 What is and is not automated

- Automated: text validation, classification, routing, entity extraction, evidence retrieval, answer composition, structured output, eligibility checks and ranking.
- Not claimed: automatic enrolment, authoritative student-record mutation, final fee/seat confirmation, or a live connection to NMIMS systems.
- Human control: low-confidence cases can ask for information, request confirmation, refer to staff, or be marked out of scope.
- Evidence discipline: facts and citations come from the project's source-derived knowledge base and verified annotations; the elective duplicate-content rule is explicitly labelled as an advisor policy rather than an NMIMS rule.

2. End-to-end architecture

The application is a Python project requiring Python >=3.11. FastAPI is the HTTP boundary, Pydantic models define strict contracts, and the reusable component in src/nlp/component orchestrates the trained pipeline. The static interface is served at /ui. The root route redirects to that UI, and /docs exposes OpenAPI documentation.

```
Browser (/ui)
    -> FastAPI routes
       -> /v1/analyze -> AdminNLP -> component pipeline
       -> /v1/electives/advise -> elective advisor
       -> /v1/electives/advise-from-partners -> partner client + advisor
       -> /partner-mock/* -> bundled stand-in contracts
    component pipeline
       -> preprocessing -> intent/OOS -> routing -> NER
       -> knowledge store -> BM25 + dense -> RRF -> MMR answer
       -> service request + confidence decision
    data/final, data/annotations, data/electives, models/
```

The browser is intentionally a thin client. It sends JSON and renders the structured response: intent, department, confidence, answer, next action, documents, extracted information, official sources and service-request JSON. Escaping is performed before rendering values into the page.

2.1 Main API surface

Table 2. Implemented HTTP routes

Route	Purpose	Input / output
POST /v1/analyze	Full administrative pipeline	AnalyzeInput -> AnalyzeOutput
POST /v1/analyze/batch	Analyze up to 64 requests	BatchInput -> BatchOutput
POST /v1/intent	Intent and routing only	IntentInput -> IntentOutput
POST /v1/entities	Entity extraction	TextInput -> EntitiesOutput
POST /v1/retrieve	Evidence passages	RetrieveInput -> RetrieveOutput
GET /v1/health	Liveness and loaded state	JSON status
GET /v1/info	Labels, limits and thresholds	ComponentInfo
POST /v1/electives/advise	Eligibility and recommendation	AdviseInput -> AdviseOutput
POST /v1/electives/parse	Parse a syllabus	SyllabusInput -> ParsedSyllabus
POST /v1/electives/advise-from-partners	Fetch partner data then advise	topic/student id + advisor inputs
GET /v1/manifest	UI/API capability manifest	JSON

3. Ask the copilot: complete request lifecycle

3.1 Request validation and limits

AnalyzeInput is a strict Pydantic model. text is required, stripped, non-blank and limited to 1,000 characters. Options are strict as well: top_k is 1–13, k_sources is 1–10, routing_review_below is 0–1, and timings can be included or omitted. BatchInput accepts 1–64 AnalyzeInput records. Extra fields are forbidden, which makes accidental contract drift visible as a 422 validation error.

```
POST /v1/analyze
Content-Type: application/json

{"text": "I lost my ID card. What should I do?",
 "request_id": "portal-123",
 "options": {"top_k": 3, "k_sources": 3, "include_timings": true}}
```

3.2 Preprocessing

The preprocessing module supports Unicode normalization, lowercasing, contraction expansion, institutional and chat abbreviation expansion, entity masking, punctuation cleanup, tokenization, optional spelling correction, optional stopword removal, and lemmatization or stemming. The default training configuration uses spelling correction and lemmatization without stopword removal. Embedding classifiers intentionally see near-raw text after only contraction and abbreviation expansion. Retrieval uses a separate BM25 preprocessor with stopword removal and lemmatization.

This separation matters: a classifier benefits from semantic wording and retained context, while lexical retrieval benefits from normalized content terms. The fallback does not run the full trained preprocessing/model stack; it uses deterministic keyword matching and the verified catalogue.

3.3 Intent classification and out-of-scope rejection

The taxonomy contains 12 top-level intents, including CERTIFICATES, TRANSCRIPTS, EXAMINATIONS, FEES, SCHOLARSHIPS, ATTENDANCE, ACADEMIC_REGISTRATION, ID_CARDS, HOSTEL, LIBRARY, STUDENT_SERVICES and STUDENT_WELFARE. Their 95 sub-intents represent specific services such as DUPLICATE_ID_CARD, TRANSCRIPT, REVALUATION and COURSE_REGISTRATION.

The production pipeline predicts a top-level class and probability. If the top probability is below the calibrated rejection threshold, the result is out_of_scope and no routing is produced. Alternatives are retained for transparency. This prevents a confident-looking administrative answer to an unrelated request.

3.4 Sub-intent routing

Routing is constrained by the predicted top-level intent rather than searching every service equally. This reduces incoherent labels—for example, an ID-card query should be compared with ID-card services before unrelated hostel services. Routing output includes the chosen sub-intent, department, confidence and alternatives. A review threshold can cause refer_to_staff when routing confidence is low.

3.5 NER and information extraction

Rule extraction recognizes EMAIL, PHONE, AMOUNT, DATE, PERCENTAGE and STUDENT_ID. The broader annotation and learned paths use BIO tagging. CRF features include token context, shape, prefixes, suffixes, bigrams and gazetteer signals; cross-fit gazetteer mode hides entity values by value-level folds to reduce memorization. The transformer NER path aligns character offsets to BIO labels, uses maximum length 96, eight epochs, learning rate 5e-5 and batch size 16, retaining the best validation span-F1 state. Rule and learned entities are merged without overlap.

3.6 Service catalogue and structured request

The verified facts in data/annotations/facts.yaml provide source-linked service metadata. load_services() keeps non-historical facts, groups them by sub-intent, deduplicates required documents, chooses the most common department and uses configured next actions. The final ServiceRequestResult contains intent, sub_intent, department, required_documents, next_action, extracted_details, missing_information, priority, status, supporting_facts and source_urls.

The status is ready, needs_information or needs_confirmation. The decision action is one of auto_route, ask_for_information, confirm_with_student, refer_to_staff or out_of_scope. This is the stretch goal made explicit as a typed object that another system can store as a draft ticket rather than parsing prose.

3.7 Knowledge-base provenance and passage units

Phase 1 downloads and inventories official sources, extracts HTML with trafilatura, extracts PDF text page by page, and OCRs pages with fewer than 15 extracted words using RapidOCR. The build step creates nmims_knowledge_base.csv. PDF knowledge units are page-level rows; HTML content is split into approximately 220-word windows. At runtime, load_chunks() creates Chunk records retaining source id, page, category, sub-intent, title, section, text, URL, fact ids and currency metadata.

Therefore, the runtime does not perform dynamic recursive request-time chunking. The precise claim is that the released knowledge base is indexed as page-level passages for PDFs and approximately 220-word windows for HTML. This preserves page citations and makes evidence traceable.

3.8 Hybrid retrieval

BM25 uses Okapi BM25 with k1=1.5 and b=0.75 over preprocessed indexed text. Dense retrieval uses sentence-transformers/all-MiniLM-L6-v2 with normalized CPU embeddings. Hybrid retrieval obtains rankings from dense and BM25 retrievers and fuses them at document level with Reciprocal Rank Fusion, score(d)=sum 1/(60+rank). The best chunk is retained per document. A predicted intent boost of 1.2 and sub-intent boost of 1.3 are applied after fusion; boosts are soft and do not erase all other candidates.

Results are then returned as ranked SourceResult objects with document id, title, page, URL, fused score and relative score. The source list is evidence for both the answer and service request.

3.9 Answer generation and RAG boundary

The default answer is extractive. Candidate sentences are collected from verified facts and retrieved passages, normalized, and restricted to sentences of 6–60 words. MMR uses normalized sentence embeddings and the query embedding, lambda 0.75, selects at most four sentences, and limits the summary to 70 words. The generator appends relevant department, required-document and next-step information from structured metadata.

Optional LLM paths do not replace retrieval. LLMSummarizer first selects evidence with embedding similarity, then prompts either local Qwen/Qwen2.5-1.5B-Instruct or Claude with an evidence-only instruction, a two-sentence/under-60-word limit, and exact copying for fees, dates, email addresses and form names. The Claude path requires credentials and is not required for the default extractive pipeline.

3.10 Confidence and operational decisions

Week 6 records calibrated confidence bands and selective actions. Saved results show that high-confidence intent cases were more accurate than medium/low cases, which is why the component exposes confidence and does not force every request into auto-routing. Timings are optional and identify per-stage latency. Errors are explicit: invalid input is 422, unavailable trained models is normally 503, and partner failures are surfaced as PartnerError rather than silently converted into successful answers.

3.11 Fresh checkout fallback

The versioned API checks whether the trained component can load. If model artifacts are absent, it injects DemoCopilot with pipeline='demo'. DemoCopilot uses deterministic keyword matching for intent/sub-intent, the verified service catalogue, page-level knowledge-base overlap for source hits and rule-based entities. It exists so the browser is demonstrable from a clean repository checkout. It is not a replacement for the trained v3 component and does not alter CLI behavior or trained evaluation.

4. Elective advisor

4.1 Inputs and file handling

AdviseInput contains one StudentInput and one to twenty SyllabusInput records. StudentInput can contain free-text identity, explicit program/year/semester overrides, completed courses and optional interests. SyllabusInput accepts text or base64-encoded PDF, DOCX, TXT or Markdown, plus a name and source label. Syllabus text is capped at 30,000 characters; student fields have explicit bounds.

read_syllabus decodes base64, extracts PDF text with PyMuPDF, reads DOCX paragraphs with python-docx, or decodes text. No external language model is required for parsing. The parser normalizes case, punctuation, degree abbreviations and programme abbreviations, and recognizes official catalogue names so a course containing “and” is not split incorrectly.

4.2 Syllabus parsing

The parser first looks for labelled fields such as Course Title, Course Type, Program(s), Year/Semester, Credits and Prerequisites. It then applies prose patterns for phrases such as

third-year/Semester VI eligibility, completed prerequisites, programme elective and three-credit courses. Each field records labelled, inferred or missing provenance. ParsedSyllabus reports course_type, programmes, open_to_all, level, semesters, credits, prerequisites, topics, is_new, fields_found and parse_confidence.

Formal outlines and unseen new-course proposals therefore share one typed representation. A new course is not treated as an official catalogue course merely because its text looks plausible; it is marked is_new and assessed against the student and published structure.

4.3 Student identity and catalogue lookup

The official catalogue is parsed from the extracted MPSTME Student Resource Book 2026 (SRC052). The saved Week 7 results contain 24 programmes, 1,474 rows and 191 elective slots. The catalogue distinguishes compulsory courses, programme elective slots, open elective slots and named electives, and records semester, year of study, credits, category, source id and page.

Identity parsing resolves degree level (UG or PG), programme alternatives, year and semesters. If only a year is given, both semesters in that year can be considered. Earlier compulsory courses in the official structure are treated as completed, and explicit completed_courses are added. An explicit program/year/semester input overrides the corresponding free-text inference.

4.4 Eligibility checks

Each syllabus receives cited checks for level, programme, semester, elective slot, first-year restriction, prerequisites and duplicate content. The level rule follows the cited CBCS statement that UG options are taken from UG programmes and PG options from PG programmes. The first-year rule follows the cited Student Resource Book text excluding first-year UG and first-term first-year PG students from CBCS courses. Programme and semester checks compare parsed syllabus fields with official catalogue rows and student profile.

Prerequisites are matched to completed courses using the shared embedding encoder plus lexical content-word overlap. A match score of at least 0.75 counts as completed, 0.60–0.75 is a warning requiring review, and lower matches fail or remain uncertain according to the assessment path. Duplicate content uses a title/content similarity policy with threshold 0.85; this is explicitly our advisor policy, not a university rule.

4.5 Ranking and output

Eligible options are ranked using prerequisite fit, novelty and optional interest match. Novelty is one minus similarity to the closest course already in the programme. The output includes one CourseAssessment per syllabus, best-first; a Recommendation; per-semester recommendations; overlap pairs; enrolment instructions; human-review items; citations; and a disclaimer that the result is advice, not enrolment.

The enrolment handoff cites course-coordinator/program-chair routing and published elective availability language. It also notes the minimum-student rule cited in the source. A production portal should show the recommendation as a draft choice and require the coordinator or programme chair to confirm offering, capacity, prerequisites and the student’s live record.

4.6 Evaluation evidence

Table 3. Week 7 elective-advisor evidence

Metric	Development	Held-out
Cases	10	13
Course decisions	48	65
Status accuracy	1.0000	0.9846
Can-enrol accuracy / precision / recall	1.0000 / 1.0000 / 1.0000	1.0000 / 1.0000 / 1.0000
Recommendation accuracy	1.0000 (9 cases)	1.0000 (12 cases)
Programme-profile accuracy	1.0000	1.0000
Semester accuracy	1.0000	1.0000

Syllabus parsing evaluated 15 cases in each of original, lowercased and labels-removed/reordered conditions. Course type, programmes, semesters, prerequisites and all-fields accuracy were 1.0 in each saved condition. The evaluation is finite and project-specific; it is not evidence that every future syllabus or university rule will parse correctly.

5. Partner integration

5.1 Inbound contracts

PartnerClient consumes two HTTP contracts. A syllabus service implements GET {PARTNER_SYLLABUS_URL}/syllabi?topic=<topic> and returns service, topic and courses[]. Each course requires course_id, title and syllabus_text. A student-record service implements GET {PARTNER_STUDENT_URL}/students/<student_id> and returns service, student_id, program, year, semester (nullable) and completed_courses.

```
GET /syllabi?topic=artificial%20intelligence
{"service":"syllabus service","topic":"artificial intelligence",
 "courses":[{"course_id":"AI101","title":"...",
             "syllabus_text":"..."}]}

GET /students/S-1001
{"service":"student records","student_id":"S-1001",
 "program":"B Tech Computer Science","year":3,
 "semester":6,"completed_courses":[]}
```

Only fields needed for advising are requested from student records: programme, year, semester and completed courses. The client does not request name, contact details or grades, supporting data minimization.

5.2 Configuration, mocks and errors

Set PARTNER_SYLLABUS_URL and PARTNER_STUDENT_URL to use real partner services. PARTNER_TIMEOUT_S defaults to 8 seconds. If a base URL is absent, the same contract is served by in-process mock functions, and /partner-mock exposes the bundled HTTP stand-in for an end-to-end demonstration. The mocks use illustrative syllabus files under data/electives/syllabi and three fictional student records.

Every call records service, URL, elapsed milliseconds, success and whether mock data was used. URL errors, timeouts, JSON errors and contract violations become PartnerError. A failed partner call is not represented as a successful recommendation.

5.3 How another university service consumes the API

A student portal can call /v1/electives/advise directly after obtaining the minimum student context from its own authenticated session. A helpdesk can call /v1/analyze and store request_id, decision, service_request, source URLs and the answer as a draft ticket. Authentication, authorization, rate limiting, audit retention and live university database writes are deployment responsibilities not implemented in this repository.

```
import requests

payload = {
    "student": {"program": "B Tech Computer Science", "year": 3,
                "semester": 6, "completed_courses": []},
    "syllabi": [{"name": "new-course.txt",
                 "text": "Course Title: Generative AI
"
                         "Course Type: Program Elective
"
                         "Program(s): B.Tech Computer Science
"
                         "Year / Semester: III / VI
"
                         "Prerequisites: Machine Learning"}]
}
response = requests.post(
    "https://copilot.example.edu/v1/electives/advise",
    json=payload, timeout=15)
response.raise_for_status()
recommendation = response.json()["recommendation"]
```

A production integration should treat the response as advisory JSON: render citations and human_review, preserve the schema_version, and require a separate enrolment confirmation transaction. It should not infer authorization from a recommendation.

6. Data provenance and dataset construction

6.1 Source collection and extraction

Phase 1 maintains source configuration, fetch logs, extraction logs and a source inventory. Official pages and PDFs are downloaded into data/sources, extracted into data/interim/extracted, and converted into data/final/source_inventory.csv and data/final/nmims_knowledge_base.csv. PDF pages with fewer than 15 text-layer words are OCRed. Each verified fact in facts.yaml includes a source id and verbatim quote, which validation checks against extracted source text.

6.2 Facts, queries, annotations and variants

Phase 2 turns curated facts and queries.yaml into labels for intent, sub-intent, department, required documents, next action, source metadata, programme, semester, user type, question type, style, ambiguity and confidence. The project documentation states that core queries were written by the team from official sources rather than collected student conversations. NER labels in this phase are silver, rule/dictionary-derived spans whose entities literally occur in the query.

Phase 3 creates up to two rule-based variants per core query: casual, abbreviation, typo, formal, polite or Hinglish. Original and variants retain the same group_id. The split is group-based and approximately 70/15/15, stratified by sub-intent, so rewordings of a test question cannot leak into training. test_core contains original held-out queries.

6.3 Validation

Phase 4 checks missing values, duplicates and near-duplicates, taxonomy membership, departments, source URLs, contradictory labels, NER spans, group leakage, normalized-text leakage, fact ids, quote support and unverified claims. Unverified rows are prohibited from carrying an answer, department or next action. No notebook files (.ipynb) exist in the repository; the reproducible evidence is in Python modules, CSV/YAML data and saved JSON reports.

7. Training and evaluation history

Table 4. Progressive implementation history

Stage	Implementation	Saved evidence / result
Week 2	Official-source dataset; group split; majority, Complement NB, TF-IDF Logistic Regression and Linear SVM baselines; CLINC150 OOS	Core-test TF-IDF intent accuracy 0.8822; macro-F1 0.8805; CLINC OOS recall 0.945
Week 3	TF-IDF, frozen MiniLM/BGE embeddings, ensembles, kNN, BM25, dense/hybrid retrieval and summarizers	Pipeline v2 intent accuracy 0.8904; sub-intent accuracy 0.7902
Week 4	BGE-small fine-tuning, joint OOS labels, SetFit comparison, transformer NER, dense/cross-encoder retrieval comparisons	Final v3 intent accuracy 0.9123; macro-F1 0.9035; OOS recall 0.961
Week 5	Inference-only bootstrap CIs, McNemar tests, ECE, risk coverage, perturbation robustness and abstention fixes	Selective decisions and robustness evidence saved in week5 results
Week 6	Reusable typed component, schema, confidence bands, actions, latency and contract validation	365 outputs validated; p50 analyze 112.9 ms; p95 202.0 ms
Week 7	Elective parsing, catalogue matching, eligibility, ranking, citations, fairness/transparency evaluation	98.46% held-out status accuracy; 12/12 held-out recommendations
Week 8	Professional UI, primary Ask the copilot landing flow, packaging, README, report, presentation and demo	72 tests passed, 5 skipped in final regression run

Week 2 model selection uses validation rather than the test set. Week 4 fine-tuning uses BAAI/bge-small-en-v1.5 with 10 epochs, encoder learning rate 8e-5, head learning rate 2e-3, batch size 32, maximum length 64, inverse-square-root class weighting, gradient clipping at 1.0 and best validation macro-F1 retention. Transformer NER uses eight epochs, learning rate 5e-5, batch size 16 and maximum length 96. Dense retrieval fine-tuning uses MultipleNegativesRankingLoss on training question/passage pairs; the cross-encoder was evaluated but the Week 4 implementation notes that its score was lower and it is not used in the final copilot.

8. Testing, metrics and final verification

8.1 Automated tests

The final regression suite passed 72 tests with 5 skipped. The focused frontend/API and fallback checks passed, Python compilation passed, git diff --check passed, and live smoke tests covered the browser, /v1/manifest, the ID-card example, partner flow and the elective flow. The fallback regression verifies that a fresh checkout can answer the ID-card example with ID_CARDS, DUPLICATE_ID_CARD, School Academic Office routing, the duplicate-ID-card next action and an official citation.

8.2 Saved component metrics

Table 5. Week 6 contract and runtime evidence

Measure	Saved value
Week 6 test set	365 core test records; 305 in-scope
Intent confidence	High band: 336 cases, accuracy 0.9435; medium: 27, accuracy 0.5926; low: 2, accuracy 0.0
Routing confidence	High: 191 cases, accuracy 0.9058; low: 110, accuracy 0.6000
Auto-route precision	0.8915
Entities	Mean 1.2137 per query; 432 learned entities; mean learned confidence 0.7503
Contract	365 outputs validated; mean JSON 2,864 bytes; maximum 4,140 bytes
Latency	Analyze p50 112.7 ms / p95 192.0 ms in latency sample; pipeline p50 112.9 ms / p95 202.0 ms

Week 5’s saved test sets include 365 core queries, 607 variants, 1,000 CLINC150 OOS queries, 96 challenge queries, 120 hand-annotated NER queries, 870 retrieval questions and 3,050 robustness cases. The final document reports these values as evaluation-set descriptions, not as claims about production traffic.

9. Installation, operation and deployment

9.1 Supported local setup

```
# uv
uv sync
uv run uvicorn src.backend.main:app --reload

# pip
python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows PowerShell:
.venv\Scripts\Activate.ps1
# Windows Command Prompt:
.venv\Scripts\activate.bat
pip install -e .
uvicorn src.backend.main:app --reload
```

Open http://127.0.0.1:8000/ for the web application, http://127.0.0.1:8000/ui/ for the UI, and http://127.0.0.1:8000/docs for OpenAPI. The supported project setup is uv sync; pip is documented for environments that require it.

9.2 Model artifacts and reproducibility

The repository snapshot does not contain trained model binaries because the models directory is gitignored. To run trained inference, the expected artifacts must be produced or supplied through the project's training/evaluation workflow. Without them, the API deliberately selects the demo fallback for browser usability. This distinction must be preserved in a submitted demonstration and in any deployment documentation.

9.3 Production integration requirements

- Put the API behind university authentication and role-based authorization; do not expose student records through unauthenticated endpoints.
- Pin model/data versions and retain the component_version and schema_version in every stored request.
- Add structured logs, latency/error dashboards, source freshness checks and alerts for partner failures.
- Use a secrets manager for external API credentials; do not place credentials in source or the README.
- Require staff confirmation before enrolment or any irreversible administrative action.
- Define retention, deletion and access policies for request text, extracted entities and partner responses.

10. Responsible AI, privacy and limitations

The system is designed to abstain rather than invent when confidence or evidence is insufficient. It cites official passages, exposes alternatives and records whether a rule is NMIMS, syllabus-stated or advisor policy. The elective output explicitly says it is not an enrolment and identifies what a coordinator must confirm.

The fairness evaluation for Week 7 reported 0 changed decisions across 230 counterfactual variants. That result applies to the evaluated variants and does not establish universal fairness. The source data is institution-specific, some sub-intents are marked unsourced, official documents can change, and OCR can introduce errors. Generated variants are not real student traffic. A production university must refresh sources, re-run validation, version policies and monitor errors.

Privacy boundaries are especially important because requests may contain student ids, emails, phone numbers, fees or dates. The component extracts these for routing but the repository does not implement a complete identity, consent, retention or access-control system. Partner integration intentionally requests only the academic fields needed for elective advice.

11. Complete implementation reference

Table 6. Module and artifact map

Path	Responsibility
src/backend/main.py	FastAPI app, static UI, root redirect, error handling
src/backend/api/v1.py	Versioned NLP routes and fallback selection
src/backend/api/electives.py	Elective and partner endpoints
src/backend/api/partner_mock.py	HTTP mock partner routes
src/backend/static/index.html / app.js / styles.css	Professional browser UI and rendering
src/nlp/component/schemas.py / engine.py / demo.py	Typed contracts, orchestration, deterministic fallback
src/nlp/pipeline/copilot.py / summarize.py	End-to-end inference and answer generation
src/nlp/retrieval/search.py / rerank.py	BM25, dense, hybrid RRF and evaluated reranking
src/nlp/knowledge/store.py	Knowledge chunks and service catalogue
src/nlp/ner/extractor.py / transformer_ner.py	Rules, CRF and transformer NER
src/nlp/electives/advisor.py / syllabus.py / catalog.py / schemas.py	Elective parsing, rules, ranking and contracts
src/nlp/integrations/partners.py / mock.py	Partner HTTP contracts and mock data
src/dataset/phase1_sources.py … phase4_validate.py	Source, annotation, expansion and validation pipeline
data/final / data/annotations / data/electives	Released passages, splits, facts, cases and illustrative syllabi
reports/week2 … week7/results.json	Saved evaluation evidence
tests/	API, NLP, component, elective, fallback and evaluation tests

12. Conclusion

The completed application connects the stated administrative NLP problem to a usable web interface and to a concrete elective-advising decision. Ask the copilot converts unstructured questions into evidence-backed, actionable service requests. Elective advisor converts heterogeneous syllabi and academic identity into cited eligibility decisions and ranked options. Partner integration defines the boundary through which a university portal, syllabus service or student-record service can participate.

The implementation is ready for demonstration and repository submission, with the important qualification that trained model files, live university integrations and final enrolment authority remain deployment concerns. The repository’s strongest production principle is therefore explicit separation: verified evidence from generated prose, model inference from demo fallback, advisory recommendations from authoritative university actions, and partner contracts from bundled mocks.